

Brew: The Self-Optimising Cafe

Master plan: product, simulation, ML/RL, data, UI, sound, and impact

October 2026 · v1.0

0. TL;DR

1. Goals, non-goals, success criteria

- 1.1 Goals
- 1.2 Non-goals for the hackathon
- 1.3 Definition of done for the demo

2. The experience: screens and interactions

- 2.1 Lobby: the cook's view (hero screen)
- 2.2 Kitchen: stations in action
- 2.3 Pantry: walk-in fridge, freezer and dry store
- 2.4 Back office: "the brain"
- 2.5 Policy Arena
- 2.6 Chaos console (adversarial demo)
- 2.7 Customer QR view (stretch)
- 2.8 Global UX rules

3. Art direction: "Strawberry Milk"

- 3.1 Palette (CSS custom properties in `tokens.css`)
- 3.2 Typography
- 3.3 Illustration style
- 3.4 Motion system

4. Sound design

- 4.1 Principles
- 4.2 Mix buses (Web Audio graph)
- 4.3 Sound list (first pass, about 60 cues)
- 4.4 Sourcing and licensing (we will download)
- 4.5 Processing pipeline (`scripts/audio_build.py` , run via `uv run`)

5. System architecture

- 5.1 Key design decisions

6. Repository layout and tooling

7. Database architecture

- 7.1 Storage strategy
- 7.2 Entity overview
- 7.3 Tables and fields
- 7.4 Indexes, integrity and invariants
- 7.5 Telemetry in Parquet (`data/runs/{world_id}/`)

8. The simulator (digital twin)

- 8.1 Engine

- 8.2 Demand generation

- 8.3 Kitchen model

- 8.4 Delivery and aggregators

- 8.5 Reputation and long-term effects

- 8.6 Real-world factors modelled

- 8.7 Synthetic history generation

- 8.8 Calibration (realism story for judges)

9. Customer personas

10. Inventory model

11. Forecasting models

- 11.1 Targets

- 11.2 Models

- 11.3 Evaluation

- 11.4 Why probabilistic matters

12. Optimisation models

13. Reinforcement learning design

- 13.1 What RL decides

- 13.2 Architecture: hierarchical, hybrid

- 13.3 Observation (manager)

- 13.4 Action space

- 13.5 Reward

- 13.6 Training pipeline

- 13.7 Explainability

14. Adversarial training ("chaos phase")

15. Bottleneck detection and investment advisor

16. Policy Arena and evaluation

- 16.1 Policies

- 16.2 Protocol

- 16.3 Presentation

17. API and real-time protocol

- 17.1 REST (FastAPI, OpenAPI auto-docs at `/docs`)

- 17.2 WebSocket `/ws/worlds/{id}`

18. Event → Visual → Sound map (the "juice" table)

19. Extra ambitious features (our additions)

20. Build plan

- 20.1 Team split (4 people; adjust to team size)

20.2 Timeline (assuming about 48 hours; scale proportionally)

20.3 De-risking rules

21. Risks and mitigations

22. Demo script (5 minutes)

23. Impact: arguments for the judges

23.1 Economic impact

23.2 Environmental impact

23.3 Social impact

23.4 Live impact scoreboard (in the Office and the closing slide)

24. Anticipated judge questions

Appendix A: Seed menu (demo cafe "brew")

Appendix B: Example scenario config

Appendix C: Policy interface

Appendix D: Glossary

0. TL;DR

Brew is an operating system for independent cafes, packaged as a cozy, playable restaurant game. It does two jobs:

1. **Runs the kitchen.** It decides which orders to accept, what to prep ahead and when, which orders to cook together (across dine-in, takeaway, Zomato and Swiggy), and how to adjust the menu (prices, featured items, hiding items) minute by minute.
2. **Tells the owner where to invest.** It answers *"What is limiting my throughput right now, and what is the most profitable thing to buy next?"* That could be another table, a second espresso machine, an extra barista at 8 a.m., or more oat milk.

A **digital twin simulator** of a cafe sits underneath both jobs. Synthetic customers with distinct personalities, staff, equipment, perishable inventory, delivery riders and reviews all live in it. **Forecasting** and **optimisation** models feed a **reinforcement-learning agent** that trains across weeks of simulated days and then through an **adversarial "chaos" phase** (sick staff, broken ovens, late suppliers, rider shortages). We prove the gain by racing four policies on identical days in a split-screen **Policy Arena**: **A** naive FCFS, **B** heuristic batching, **C** constraint solver, **D** learned RL. A fifth, an **E** perfect-information oracle, shows the upper bound.

The front end is a pink 2.5D cafe in the spirit of a restaurant game. It has a ticket rail of paper orders with rendered food photos, a thermal bill printer, a live-updating menu book whose pages you can turn, an opening fridge, kitchen stations, a split-flap "Now Brewing" board, and brown "brew"-stamped delivery bags that riders pick up. Every state change has a smooth transition and a sound.

Stack: HTML/CSS/JS (native ES modules plus Web Components, so it ports cleanly to Next.js later) · Python managed with **uv** · FastAPI + WebSockets · PostgreSQL (operational) + Parquet/DuckDB (simulation and RL telemetry) · custom discrete-event simulator · LightGBM quantile forecasting · OR-Tools CP-SAT/MILP · Gymnasium + Stable-Baselines3 (MaskablePPO / RecurrentPPO) · Howler.js + Web Audio · GSAP.

1. Goals, non-goals, success criteria

1.1 Goals

#	Goal	How we show it
G1	Show that a learned policy earns more profit than naive, heuristic and solver baselines	Policy Arena: paired seeds, 95% CIs, live race
G2	Make the AI's decisions <i>visible and understandable</i>	Every decision shows up as an animation, a sound and a "why" note
G3	Stay robust to real-world chaos	Adversarial phase; profit CVaR under disruptions
G4	Turn profit into growth	Bottleneck board + investment shop with ROI and payback
G5	Deliver measurable social, environmental and economic impact	Live triple-bottom-line scoreboard
G6	Look and sound like a polished game	Art bible, motion rules, mixed and mastered sound

1.2 Non-goals for the hackathon

- Real Zomato/Swiggy integration. Their partner APIs are gated, so we build **adapters with the same interface** and drive them from the simulator.
- Real payments, real accounts, or real customer PII.
- Native mobile apps. We do responsive web only, plus an optional customer QR view.

1.3 Definition of done for the demo

- One click starts a simulated day. Customers walk in, order, sit, eat and leave. Tickets hang on the rail, food gets cooked, bills print and bags get picked up.
- The menu book's prices change live, with an animated reason.
- The Arena races A/B/C/D on the same day, and the profit gap is visible and statistically backed.
- Pressing a "chaos" button (e.g. *barista calls in sick*) makes the RL agent visibly adapt.
- The bottleneck board names the binding constraint. Buying the recommended upgrade with earned profit raises throughput in the next simulated day.

2. The experience: screens and interactions

The app is one canvas made of "rooms". Moving between rooms uses a camera move (View Transitions API plus a GSAP pan), not a page reload. A persistent **HUD** at the top holds the clock, weather, cash, today's profit, rating stars, speed controls (1x 10x 60x), active policy and a sound mixer.

2.1 Lobby: the cook's view (hero screen)

- **Composition (front to back):** a wooden counter across the bottom third (cook's POV) → the **ticket rail** hanging above it → the dining room behind: tables, chairs, windows, plants, door → windows to the street showing weather and time of day.
- **Customers** are layered SVG "paper-doll" sprites (body, skin tone, hair, outfit, accessory) for variety, tinted by persona. Each has a state machine: `enter → queue → order → wait (patience ring) → seated → eating → linger → pay → leave`. Laptop campers open a laptop. Groups push tables together.
- **Patience ring:** a thin ring around the head that drains from pink through amber to red. When it empties, the customer either reneges (walks out with a "huff" sound and a grey cloud) or stays and leaves a bad review (a star floats to the HUD rating and cracks).
- **Ticket rail:** each order is a paper ticket clipped to a steel rail. A ticket shows:
 - the order no., channel stamp (DINE-IN / TAKEAWAY / ZOMATO / SWIGGY drawn as generic coloured stamps, **not** brand logos), and a countdown to promise time
 - a **rendered photo** for each item, with **customisation overlays** (e.g. an "oat" milk badge, "extra shot" bean icon, "no onion" crossed chip) and handwritten notes in a script font
 - a persona hint icon (briefcase = commuter, books = student, cup-and-saucer = leisurely)
 - Tickets **slide in, re-order with FLIP** when priority changes, get **clipped together with a pink paperclip** when batched ("Batch x3"), and get **torn off** with a paper sound when served.
- **Bill printer:** a thermal printer on the counter. On payment it prints the receipt line by line (paper grows and curls, monospace font, itemised GST, a QR) with a matching print sound. The receipt flutters off into a spike.
- **Menu book:** a stand-up book on the counter. Click to open full screen and turn pages (page-flip physics with a page sound). Pages are categories. Prices are **live**. When the RL agent changes a price, the old price is struck through in ink and the new one is hand-written in. A tiny ↑/↓ chip with the reason ("rush hour · kitchen at 92%") fades in, and a stamped "Today's Pick" sticker appears on featured items. Hidden (86'd) items get a "Sold out for now" ribbon.
- **Delivery shelf:** brown kraft paper bags with a pink **"brew"** stamp line up on a shelf at the end of the counter, each tagged with its rider ETA. A rider silhouette appears at the door, the bag gets picked up, and it fades out with a scooter sound. Bags waiting too long show a little steam fading, because food quality decays.
- **"Now Brewing" board:** a retro **split-flap (Solari) board** above the counter showing *Now Brewing / Almost Ready / Ready for Pickup* with flap-clatter sound. This is the customer-facing "screen of orders being made".

2.2 Kitchen: stations in action

A side-scrolling kitchen line, one station per piece of equipment:

Station	Visual	Interaction cues
Espresso machine (2–3 group heads)	Portafilters lock in, crema pours into cups, steam wand frothing pitcher	grind, tamp, pour, steam hiss
Grinder	Bean hopper level = inventory	burr whirr

Station	Visual	Interaction cues
Panini press / grill (4 slots)	Lid closes, grill marks appear, timer dial	sizzle, lid clunk
Oven (deck, 2 trays)	Window glow, croissants rise, door opens	fan hum, ding
Fryer	Basket dunk, bubbles	sizzle crescendo
Blender	Spinning vortex of pink smoothie	blender ramp
Cold station / cold brew tower	Drip animation, tower level shows prepped batch	drip
Prep board	Knife chop sprites, prepped containers appear in the "mise en place" rail	chop
Pass / plating	Plates slide to the pass, bell	"order up" bell
Dish pit	Stack of dirty cups (can be the bottleneck!)	clinks, dishwasher whoosh

- Each slot shows a **progress ring**, a staff avatar and the order no. Batched items share a single coloured outline.
- **Staff** avatars move between stations and show a fatigue meter that matters in the simulation. They take breaks, which are enforced.
- **Broken equipment** (chaos) sparks, gets a hazard-tape overlay and an "Out of order" sign, and a technician walks in after the MTTR.

2.3 Pantry: walk-in fridge, freezer and dry store

- Fridge doors **swing open in 3D** (CSS `perspective` and `rotateY`) with a fridge-seal "thunk" and a cold-air puff of particles and hum.
- Shelves hold item containers (milk cartons, cream, paneer, berries, avocado, croissant dough trays). Each shows:
 - **Fill level** (liquid level / count) and the **number of lots** (stacked labels)
 - **Freshness colour** per lot: fresh mint → amber → expiring red, plus a FEFO "use first" arrow
 - Opened vs sealed state (an opened carton has a folded spout)
- Clicking an item opens a **lot card**: lots, received and expiry dates, opened time, remaining shelf life, forecast usage (P50/P90) for today, days of cover, reorder suggestion, CO₂e/kg and supplier.
- A **dry store shelf** holds beans, flour, sugar and packaging. **Packaging is inventory too**: bags, cups, lids.
- **Deliveries**: a supplier van pulls up outside, boxes slide into place, and the stock counters tick up.
- **Waste bin** and a **donation crate**: near-expiry items get marked down ("Happy Hour" on the menu), and leftovers go to a donation crate at close. Both are counted on the impact board.

2.4 Back office: "the brain"

A cozy manager's office with a corkboard, laptop and window: - **Forecast board**: demand fan charts (P10–P90) per item and channel for the next 4 hours, and actual-vs-forecast lines. - **Decision feed**: sticky notes from the agent, e.g. *"Pre-prepping 6 cold brews: P85 demand at 1–2 pm is 7, current stock 1."* Each note links to the decision and shows its top contributing factors. - **Bottleneck board**: a "What's limiting us?" gauge plus a ranked list (\$15). - **Investment shop**: a catalogue of items (table, chair set, second espresso machine, bigger fridge, hire a barista for the morning shift, a marketing push) showing **expected Δprofit/day, payback days and confidence**. Buying spends the cash earned in the sim, and the item **physically appears** in the lobby or kitchen with a "delivery" animation. - **Ledger**: P&L for today, this week and lifetime, with a waterfall chart (revenue → COGS → commission → labour → waste → energy → profit).

2.5 Policy Arena

- **Four mini-cafes in a 2×2 grid** (A, B, C, D, with E shown as a dotted ghost line on the chart). They are fed the **same customer arrivals and the same disruptions** (common random numbers).
- Live counters per cafe: profit, avg/P95 wait per channel, walkouts, rating, waste kg.
- A race chart of cumulative profit across the bottom. At day end a podium animation plays with confetti for the winner.
- **"Run 30 seeds"** button: runs in the background and fills a results table with means and bootstrap CIs.

2.6 Chaos console (adversarial demo)

A drawer of big pink buttons that judges can press: **Barista sick**, **Oven breaks**, **Milk delivery late**, **Rain storm (delivery surge)**, **Cricket final (delivery spike)**, **Rider shortage**, **Power cut 15 min**, **Bus of tourists**. The world reacts and the agent visibly re-plans: tickets reshuffle, aggregator throttle switches, menu items get 86'd, and pre-prep kicks in.

2.7 Customer QR view (stretch)

A phone-sized page showing the live menu with prices, an ETA estimate and the "Now Brewing" board, the way a real customer would see it.

2.8 Global UX rules

- **Nothing pops.** Every change animates: enter, exit, move and value change. Numbers tween with a rolling-odometer effect.
- **Time controls:** pause, 1x, 10x, 60x. At high speed, minor sounds are suppressed and animations compress.
- **Replay / time machine:** a scrubber over the event log lets you rewind any day.
- `prefers-reduced-motion` and a "Calm mode" (no music, softer SFX) are respected. All text meets 4.5:1 contrast. Everything is keyboard navigable.

3. Art direction: "Strawberry Milk"

3.1 Palette (CSS custom properties in `tokens.css`)

Token	Hex	Use
--pink-50	#FFF5F8	background paper
--pink-100	#FFE4EE	panels
--pink-300	#FFB30B	highlights, rings
--pink-500	#F2709C	primary brand, buttons
--pink-700	#C2457A	pressed, headings
--berry-900	#5A1E3A	dark text alternative
--cocoa-700	#5B3A29	wood, counter, text
--cream-100	#FFF8EC	tickets, receipts
--kraft-400	#C89F72	delivery bags
--mint-400	#7FD1B9	fresh / positive
--butter-300	#FFD98A	warning / amber
--cherry-500	#E5484D	critical / expired

The theme ships in **light mode (day)** and **dusk mode** (warmer, with lamps glowing). Lighting follows the simulation clock: a CSS gradient overlay on the window plus a soft multiply layer.

3.2 Typography

- Display: **Fraunces** (soft, warm serif) for headings and the menu book
- UI: **Nunito** / **DM Sans**
- Handwritten tickets and price edits: **Caveat**
- Receipts and the split-flap board: **Space Mono** / **VT323**

3.3 Illustration style

- Cozy 2.5D flat illustration with soft shadows, rounded shapes and slight grain texture. The references we are aiming for: *Good Pizza*, *Great Pizza*, *Unpacking*, *Coffee Talk*, *Overcooked*.

- **Food photos:** one consistent, "studio-rendered" look: 3/4 angle, soft pink backdrop light, transparent background, 1024 px, saved as WebP. Generate each item with a **single prompt template** (same camera, lighting and plate) using an image model, then cut out backgrounds. Customisation overlays are small separate PNGs composited on the ticket.
- **Customers and staff:** layered SVG paper dolls. About 6 bodies × 8 hairstyles × 10 outfits × 4 accessories gives thousands of combinations. Animation uses CSS transforms (bob, walk cycle via 2–4 frames).
- **Asset budget:** about 25 food renders, about 40 SVG props, 1 lobby background, 1 kitchen background, 1 pantry, 1 office.

3.4 Motion system

- **GSAP** (free, including plugins) for timelines and camera moves. The **Web Animations API** handles micro-interactions. **FLIP** handles list re-ordering (tickets, flap board).
- **Easing:** `back.out(1.4)` for things arriving, `power2.inOut` for camera, springs for paper.
- **Durations:** micro 120–180 ms, standard 250–400 ms, camera 600–900 ms.
- One **animation queue per entity**, so rapid events at 60× speed coalesce instead of stacking.
- Use a **View Transitions API** cross-fade between rooms, plus a camera pan.

4. Sound design

4.1 Principles

1. **Every event has a sound, but the mix stays calm.** A cafe is a soundscape, not an arcade.
2. **Variation beats repetition.** Each SFX has 3–5 variants, ±3% pitch jitter and ±2 dB gain jitter.
3. **Space.** Sounds are panned by on-screen x position (`StereoPanner`) and attenuated by depth (lobby far vs counter near).
4. **Adaptive.** The ambience crowd-murmur layer is scaled by **real occupancy**. Music adds layers as the kitchen gets busier, and returns to a soft bed when quiet. Rain on the window plays when the simulated weather is rainy.
5. **Rate limiting** per sound (e.g. at most 4 receipt prints per second at 60×) keeps it from becoming noise.

4.2 Mix buses (Web Audio graph)

```
sources → [ui] [sfx-kitchen] [sfx-lobby] [ambience] [music]
           → per-bus GainNode → compressor (gentle) → master gain → destination
music bus: sidechain-style ducking (-4 dB) under "important" cues (order up, chaos alarm)
```

Bus	Target loudness	Notes
Music	about -30 LUFS ("mostly inaudible")	lo-fi / soft jazz / bossa, 60–80 BPM, seamless loops, 3 stems for adaptivity
Ambience	about -28 LUFS	cafe murmur (3 density layers), fridge hum in pantry, street outside
SFX	about -20 LUFS peaks	kitchen and lobby events
UI	about -22 LUFS	clicks, paper, page turns

The mixer panel in the HUD has sliders per bus and a mute. Settings persist in `localStorage` .

4.3 Sound list (first pass, about 60 cues)

- **Lobby:** door chime, footsteps (wood), chair scrape, cutlery clink, cup on saucer, laughter (sparse), sigh / huff (walkout), "ding" on review star, cash drawer, card tap beep.
- **Counter:** ticket print (short), ticket clip snap, paper tear, paperclip "batch" click, **bill printer** (feed + cut), receipt spike, menu page turn, ink scribble (price change), stamp thump ("Today's Pick").
- **Delivery:** kraft bag crinkle, stamp thump ("brew"), scooter arrive / leave, aggregator new-order chime (**our own generic chime**, never the platforms' real ones).
- **Kitchen:** grinder, tamp, espresso pour, steam wand, milk froth, panini lid, sizzle loop, oven door, oven ding, fryer drop and bubble loop, blender ramp, knife chop, pass bell ("order up"), dishwasher whoosh, cup stack clink.

- **Pantry:** fridge door open / close (seal thunk), compressor hum loop, freezer frost crackle, box slide, van engine and door.
- **Office / meta:** split-flap clatter, coin "ka-ching" when profit milestones are hit (pitch rises with size), purchase "delivery" fanfare (soft marimba), chaos alarm (gentle but clear), podium fanfare.

4.4 Sourcing and licensing (we will download)

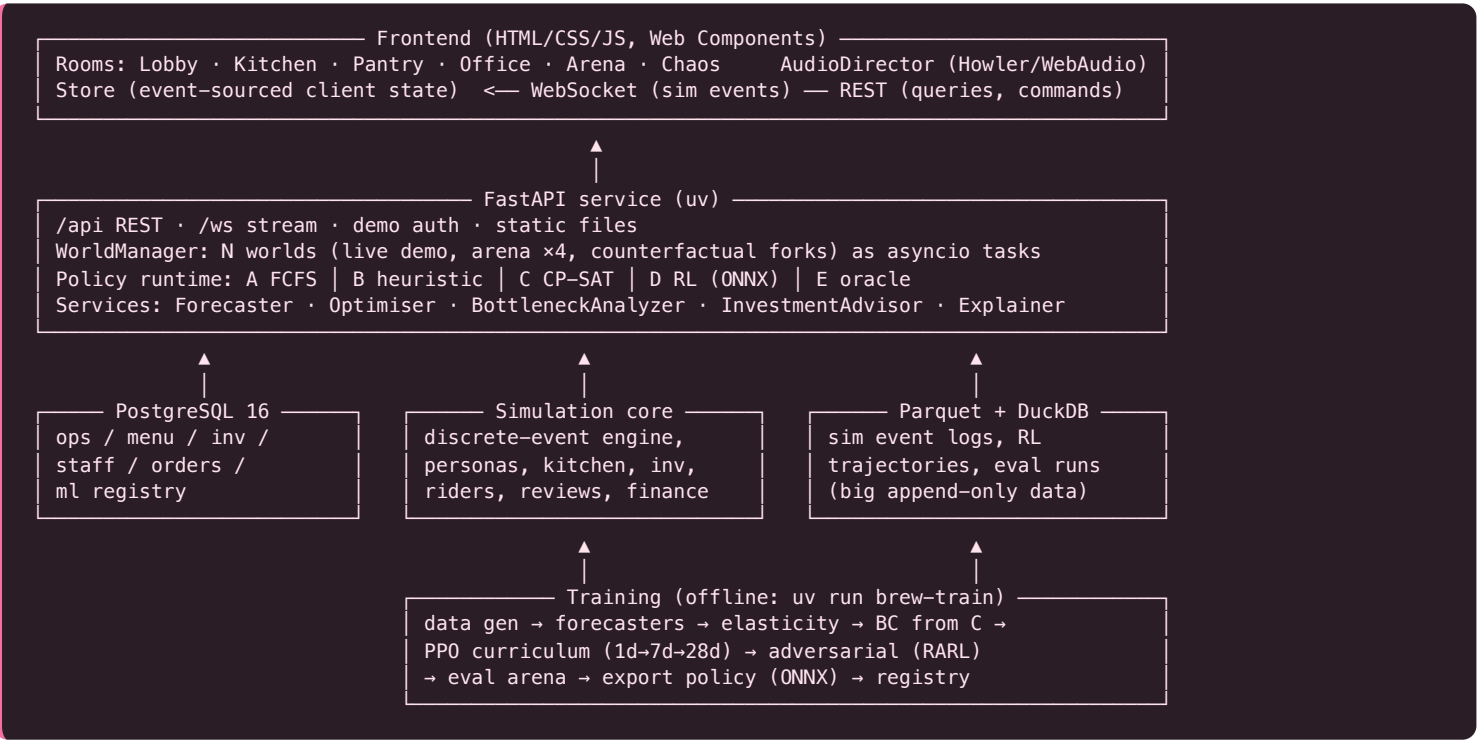
- **Freesound.org:** filter to **CC0** only (no attribution friction).
- **Kenney.nl** audio packs (UI, impact, RPG): **CC0**.
- **Pixabay** music and SFX (Pixabay Content License, free for commercial use, no attribution).
- **Sonniss GDC Game Audio Bundles** (royalty-free, commercial OK).
- Optional: generate any missing one-off SFX with a text-to-SFX model.
- Every file is recorded in `web/assets/audio/CREDITS.md` with source URL, author, license and date.

4.5 Processing pipeline (`scripts/audio_build.py` , run via `uv run`)

1. Trim silence, fade edges, convert to mono for SFX and stereo for music/ambience.
2. Normalise with `ffmpeg loudnorm` to per-bus targets.
3. Encode **Opus/WebM** + **AAC/M4A** fallback.
4. Pack small SFX into **audio sprites** (Howler.js sprite JSON) to cut HTTP requests.
5. Output a `sounds.json` manifest: `{id, file, sprite, variants, bus, gain, rateLimit, pan:"auto"}`.

Runtime: **Howler.js** for loading, sprites and fallbacks, plus a thin `AudioDirector` that maps simulation events to cues (§18). Audio unlocks on the first user click, as browsers require.

5. System architecture



5.1 Key design decisions

1. **One simulation engine, two clocks.** *Headless mode* runs as fast as possible (training, arena seeds, counterfactuals). *Live mode* paces sim time against wall-clock time × speed and streams events to the UI. The same code runs in both, so what judges see is what the agent trained on.
2. **The "world" abstraction.** Every operational row carries a `world_id`. A world is a real cafe, a live demo simulation, an arena lane or a counterfactual fork. One schema serves real operations *and* simulation, so moving to production is a configuration change.

3. **Event sourcing.** The simulator emits an append-only event stream. The UI, the database writer, the KPI aggregator and the replay scrubber are all consumers of that stream.
4. **Forkable state.** Simulator state can be snapshotted and copied (`world.fork()`). Counterfactuals like "what if we had one more table" run from the *current* state, not from scratch.
5. **Hybrid AI. RL decides and parametrises;** the forecaster and solver **inform and execute**. RL does not re-learn arithmetic that a solver does exactly. This is faster to train, more robust and easier to explain.
6. **Policy as a plugin.** Every policy implements `Policy.act(observation, decision_point) -> Action` , so the arena, the live demo and the evaluation share one interface.
7. **Front end portability.** We use Web Components, an ES-module store and CSS tokens with no framework lock-in. Each component maps 1:1 onto a React component when we move to Next.js. OpenAPI generates TypeScript types later.

6. Repository layout and tooling

```

brew/
├── plan.md / plan.pdf
├── pyproject.toml           # uv project (Python 3.12+)
├── uv.lock
├── docker-compose.yml      # postgres:16 (+ optional pgadmin)
├── alembic.ini / migrations/
├── src/brew/
│   ├── config/            # pydantic-settings, YAML loaders
│   ├── db/               # SQLAlchemy 2.0 models, session, repositories
│   ├── domain/           # pure dataclasses: Order, Recipe, Lot, Staff, ...
│   ├── sim/
│   │   ├── engine.py      # event heap, clock, RNG streams, fork()
│   │   ├── arrivals.py    # NHPP arrivals, persona sampling, groups
│   │   ├── choice.py      # MNL menu choice, price elasticity, balk/renege
│   │   ├── kitchen.py     # stations, equipment slots, staff, task graph, batching
│   │   ├── inventory.py   # lots, FEFO, spoilage, prep items, suppliers
│   │   ├── delivery.py    # aggregator orders, rider ETA, handoff
│   │   ├── reviews.py     # satisfaction → rating → reputation → demand
│   │   ├── finance.py     # ledger
│   │   ├── disruptions.py # scenario + adversary events
│   │   └── scenarios/*.yaml
│   ├── forecast/         # features, LightGBM quantile, baselines, backtest
│   ├── opt/              # CP-SAT scheduler, pricing MILP, newsvendor, (s,S), LP duals
│   ├── policies/         # A_fcfs.py, B_heuristic.py, C_solver.py, D_rl.py, E_oracle.py
│   ├── rl/               # gym env, wrappers, reward, curriculum, adversary, train/eval
│   ├── analysis/         # bottleneck, counterfactual, investment advisor, KPIs, impact
│   ├── api/              # FastAPI app, routers, ws, schemas
│   └── cli.py             # brew-api, brew-sim, brew-train, brew-eval, brew-seed
├── data/                  # gitignored: parquet runs, model artifacts
├── web/
│   ├── index.html
│   ├── css/ tokens.css base.css rooms/*.css components/*.css
│   ├── js/
│   │   ├── store.js net.js router.js clock.js
│   │   ├── audio/ director.js mixer.js sounds.json
│   │   ├── rooms/ lobby.js kitchen.js pantry.js office.js arena.js chaos.js
│   │   └── components/ brew-ticket.js brew-printer.js brew-menubook.js
│   │                       brew-flapboard.js brew-bag.js brew-customer.js brew-station.js
│   │                       brew-fridge.js brew-chart.js brew-odometer.js
│   └── assets/ img/ food/ svg/ audio/ fonts/
├── scripts/               # audio_build.py, gen_food_images.py, seed_demo.py
└── tests/                 # pytest: sim invariants, inventory conservation, policy contracts

```

Python dependencies (via `uv add`): `fastapi`, `uvicorn[standard]`, `sqlalchemy>=2`, `alembic`, `psycopg[binary]`, `pydantic-settings`, `numpy`, `polars`, `duckdb`, `pyarrow`, `lightgbm`, `statsforecast` (baselines), `ortools`, `gymnasium`, `stable-baselines3`, `sb3-contrib` (MaskablePPO, RecurrentPPO), `torch`, `onnxruntime`, `optuna`, `shap` (explanations), `pyyaml`, `orjson`, `rich`, `typer`. Dev dependencies: `pytest`, `ruff`, `mypy`, `hypothesis`.

Commands:

```

uv sync
docker compose up -d db && uv run alembic upgrade head
uv run brew-seed --cafe demo           # menu, recipes, staff, equipment, suppliers, personas
uv run brew-sim --days 60 --policy A  # generate synthetic history
uv run brew-train forecast | elasticity | bc | ppo --curriculum | adversarial

```

```
uv run brew-eval --policies A,B,C,D,E --seeds 30 --days 14
uv run brew-api # serves API + /web on :8000
```

Front end libraries are vendored into `web/vendor/` so the demo works offline: Howler.js, GSAP, StPageFlip (menu book), uPlot (fast charts).

7. Database architecture

7.1 Storage strategy

Store	What	Why
PostgreSQL 16	Master data (menu, recipes, ingredients, suppliers, staff, equipment, personas, scenarios) and operational data for live and demo worlds (orders, lots, shifts, decisions, KPIs)	Relational integrity, JSONB for flexible configs, transactional, matches production
Parquet on disk + DuckDB	Raw sim event logs for training and eval worlds, RL trajectories, per-minute telemetry (staff/equipment state), arena results	Millions of rows; columnar; zero-ops; fast analytics
Model registry (Postgres table + files in <code>data/models/</code>)	Forecasters, elasticity models, RL policies (<code>.zip</code> / <code>.onnx</code>)	Versioning and lineage

Postgres schemas: `core`, `menu`, `inv`, `staff`, `ops`, `fin`, `ml`, `sim`. We use UUIDv7 primary keys (time-ordered) and `timestampz` everywhere. For sim worlds, `ts` is *simulated* time. Money is stored as `numeric(12,2)` in INR. Quantities are `numeric(12,3)` in the ingredient's base unit.

7.2 Entity overview

```
cafe ← world > scenario
|
|   ↳ sim_event, policy_decision, disruption, daily_kpi, bottleneck_snapshot
|   ↳ order > order_item > order_item_modifier
|   ↳ kitchen_task > cook_batch
|   ↳ delivery_handoff, receipt, payment, review
menu_item > recipe > recipe_component > ingredient | prep_item
|
|   ↳ recipe_step (station, equipment_type, durations, batchable)
|   ↳ modifier_group > modifier_option (recipe deltas)
ingredient > supplier_item > supplier ; ingredient ← stock_lot > stock_movement
staff ← shift ; equipment_type ← equipment ← equipment_event ; dining_table ← table_session
persona ← customer ← party - order
```

7.3 Tables and fields

`core` (organisation and worlds)

```
cafe(id uuid pk, name text, timezone text, currency char(3) default 'INR',
  address text, lat numeric, lon numeric, opening_hours jsonb, -- {"mon":["07:00","22:00"]},...}
  gst_rate numeric(5,4) default 0.05, rent_per_day numeric(12,2),
  seating_area_m2 numeric, kitchen_area_m2 numeric, created_at timestampz)

scenario(id uuid pk, key text unique, name text, description text,
  weather_profile jsonb, calendar_events jsonb, -- holidays, festivals, matches, exam weeks
  persona_mix_override jsonb, disruption_schedule jsonb,
  demand_multiplier numeric, is_adversarial bool, created_at timestampz)

world(id uuid pk, cafe_id fk, kind text check (kind in
  ('live','demo','arena','train','eval','counterfactual','replay')),
  scenario_id fk null, policy_id fk null, parent_world_id fk null, -- forks
  fork_at_ts timestampz null, seed bigint, sim_start timestampz, sim_days int,
  speed numeric, status text, cash_start numeric(12,2), reputation_start numeric,
  created_at timestampz, finished_at timestampz, notes text)

channel(id smallint pk, key text unique, -- dine_in, takeaway, zomato, swiggy, own_app
```

```
display_name text, commission_pct numeric(5,4), fixed_fee numeric(8,2),
payment_gateway_pct numeric(5,4), packaging_required bool,
default_prep_sla_s int, rating_weight numeric, accepts_throttle bool,
acceptance_timeout_s int) -- aggregator auto-cancels if not accepted in time
```

menu

```
menu_category(id pk, cafe_id fk, name, sort_order int, page_no int, icon text)

menu_item(id uuid pk, cafe_id fk, category_id fk, sku text unique, name, short_desc, long_desc,
image_url, image_alt, base_price numeric(10,2), min_price numeric(10,2), max_price numeric(10,2),
price_step numeric(6,2) default 5, primary_station text, is_active bool,
tags text[], is_veg bool, is_vegan bool, is_gluten_free bool, allergens text[],
calories int, co2e_g numeric, -- computed from recipe
hold_time_s int, -- max time between ready and handoff
quality_half_life_s int, -- quality decay after ready (fries short, croissant long)
deliverable bool, channel_overrides jsonb, -- per-channel price/availability
launched_at date, retired_at date)

modifier_group(id pk, menu_item_id fk, name, min_select int, max_select int, required bool, sort_order int)
modifier_option(id pk, group_id fk, name, price_delta numeric(8,2), extra_prep_s int,
recipe_delta jsonb, -- [{"ingredient_id": "...", "qty": +30, "uom": "ml"}, {"replace": {"from": "milk", "to": "oat_milk"}}]
overlay_icon text, is_default bool, is_available bool)

bundle(id pk, cafe_id fk, name, items jsonb, price numeric(10,2), active_from, active_to, created_by text)

price_history(id uuid pk, world_id fk, menu_item_id fk, channel_id fk null, price numeric(10,2),
effective_from timestamptz, effective_to timestamptz null,
set_by text check (set_by in ('owner', 'policy', 'rule')), decision_id fk null, reason text)

menu_intervention(id uuid pk, world_id fk, menu_item_id fk null, bundle_id fk null,
kind text check (kind in ('price', 'feature', 'hide_86', 'bundle', 'happy_hour', 'markdown_expiry')),
params jsonb, starts_at, ends_at, decision_id fk null, outcome jsonb)
```

Recipes (bill of materials and process)

```
recipe(id pk, menu_item_id fk null, prep_item_id fk null, version int, is_current bool,
yield_qty numeric, yield_uom text, notes)

recipe_component(id pk, recipe_id fk, ingredient_id fk null, prep_item_id fk null,
qty numeric(12,3), uom text, waste_factor numeric(5,4), -- trim/spill loss
is_optional bool, substitutes jsonb) -- allowed fallbacks when out of stock

recipe_step(id pk, recipe_id fk, seq int, name text, station_type text, equipment_type_id fk null,
duration_mean_s numeric, duration_sd_s numeric, duration_dist text default 'lognormal',
staff_attention numeric(3,2), -- 1.0 = hands-on, 0.1 = oven baking (staff free)
skill_key text, min_skill numeric(3,2),
batchable bool, max_batch int, batch_time_factor numeric, -- time(n) = t * (1 + f*(n-1))
depends_on int[], can_prep bool)
```

inv (inventory, beyond "number of items")

```
ingredient(id uuid pk, cafe_id fk, name, category text, -- dairy, produce, bakery, dry, beverage_base,
packaging -- g, ml, pc
base_uom text,
storage_zone text check (storage_zone in ('ambient', 'chilled', 'frozen')),
shelf_life_unopened_h int, shelf_life_opened_h int,
perishability_class char(1), -- A highly perishable ... D stable
quality_curve jsonb, -- quality vs age, for degradation
density_g_per_ml numeric null, allergen_flags text[],
co2e_kg_per_kg numeric, is_packaging bool,
par_level numeric, reorder_point numeric, safety_stock numeric,
avg_unit_cost numeric(12,4), price_volatility numeric)

uom_conversion(ingredient_id fk, from_uom text, to_uom text, factor numeric, primary key(ingredient_id, from_uom,
to_uom))

prep_item(id uuid pk, cafe_id fk, name, recipe_id fk, base_uom, storage_zone,
hold_time_min int, quality_half_life_min int, batch_size_default numeric,
prep_lead_time_min int) -- e.g. cold brew concentrate 720 min, dough proof 90 min

storage_location(id pk, world_id fk, name, zone, equipment_id fk null, capacity_l numeric,
target_temp_c numeric, current_temp_c numeric, x int, y int)
```

```

supplier(id pk, cafe_id fk, name, contact jsonb, lead_time_mean_h numeric, lead_time_sd_h numeric,
  on_time_rate numeric, fill_rate numeric, min_order_value numeric, delivery_days int[],
  cutoff_time time, is_local bool, distance_km numeric)

supplier_item(id pk, supplier_id fk, ingredient_id fk, pack_size numeric, pack_uom text,
  price numeric(12,2), moq_packs int, valid_from date, valid_to date)

purchase_order(id uuid pk, world_id fk, supplier_id fk, status text, created_by text, decision_id fk null,
  ordered_at, expected_at, received_at, total_cost numeric(12,2))
purchase_order_line(id pk, po_id fk, supplier_item_id fk, packs int, unit_price numeric,
  received_packs int, rejected_packs int, reject_reason text)

stock_lot(id uuid pk, world_id fk, ingredient_id fk null, prep_item_id fk null, location_id fk,
  po_line_id fk null, prep_batch_id fk null,
  qty_initial numeric(12,3), qty_remaining numeric(12,3), uom text, unit_cost numeric(12,4),
  received_at, opened_at null, expires_at, -- recalculated on open: min(exp, opened +
  shelf_life_opened)
  quality numeric(4,3), -- 1.0 fresh → 0
  status text check (status in ('sealed','opened','depleted','expired','discarded','donated'))

stock_movement(id uuid pk, world_id fk, lot_id fk, ts timestampz, delta_qty numeric(12,3),
  reason text check (reason in ('receive','consume','prep_in','prep_out','waste_expiry','waste_spoil',
  'waste_error','remake','transfer','donate','count_adjust','theft_shrink')),
  order_item_id fk null, kitchen_task_id fk null, unit_cost numeric, value numeric(12,2))

prep_batch(id uuid pk, world_id fk, prep_item_id fk, qty numeric, triggered_by text, decision_id fk null,
  planned_at, started_at, ready_at, expires_at, consumed_qty numeric, wasted_qty numeric)

inventory_count(id pk, world_id fk, ts, counted_by fk staff, lines jsonb, variance_value numeric)

```

staff

```

staff(id uuid pk, cafe_id fk, display_name, role text, -- barista, cook, cashier, runner, dishwasher, manager
  hourly_wage numeric(8,2), overtime_multiplier numeric, skills jsonb, --
  {"espresso":0.9,"grill":0.6,"cashier":0.8}
  speed_multiplier numeric, error_rate numeric, experience_days int,
  learning_rate numeric, fatigue_rate numeric, recovery_rate numeric,
  max_hours_week int, preferred_shifts jsonb, avatar jsonb, hired_at date, active bool)

shift(id uuid pk, world_id fk, staff_id fk, planned_start, planned_end, actual_start, actual_end,
  status text check (status in ('scheduled','present','late','absent','sick','left_early')),
  break_minutes_planned int, break_minutes_taken int, station_assignment text,
  labour_cost numeric(10,2), created_by text)

staff_break(id pk, shift_id fk, start_ts, end_ts, kind text)

```

Minute-by-minute staff state (location, current task, fatigue) is high-volume, so it goes to **Parquet telemetry**, not Postgres.

Equipment and space

```

equipment_type(id pk, key, name, station_type, slots int, -- espresso groups=2, press=4, oven trays=2
  warmup_s int, energy_kw_active numeric, energy_kw_idle numeric,
  mtbf_h numeric, mtrr_h numeric, purchase_price numeric(12,2),
  maintenance_cost_per_month numeric, footprint_m2 numeric, lifespan_years numeric, sprite text)

equipment(id uuid pk, world_id fk, type_id fk, label, status text check (status in
  ('ok','degraded','down','maintenance','off')),
  condition numeric(4,3), purchased_at, last_service_at, x int, y int, investment_id fk null)

equipment_event(id uuid pk, equipment_id fk, ts, kind text, -- failure, repair, warmup, maintenance, power_loss
  severity text, cost numeric, cause text, disruption_id fk null)

station(id pk, world_id fk, type text, name, max_staff int, equipment_ids uuid[], x int, y int)

dining_table(id uuid pk, world_id fk, label, seats int, shape text, is_outdoor bool,
  combinable_with uuid[], x int, y int, status text, investment_id fk null)

table_session(id uuid pk, table_id fk, party_id fk, seated_at, food_served_at, left_at,
  cleaned_at, cleaned_by fk staff null)

```

Demand: personas, customers, parties

```
persona(id pk, key text unique,           -- commuter, student, leisurely, remote_worker, family, office_bulk, tourist,
       regular, delivery_home, late_night
       name, description, icon,
       arrival_profile jsonb,              -- weights by dow × 15-min slot
       group_size_dist jsonb, channel_mix jsonb,
       price_elasticity numeric, price_reference_memory_days int,  -- fairness: compares to past prices
       patience_mean_s numeric, patience_sd_s numeric, patience_by_channel jsonb,
       dwell_mean_min numeric, dwell_sd_min numeric, laptop_prob numeric,
       basket_affinity jsonb,              -- item/category utilities
       add_on_prob numeric, quality_sensitivity numeric, temp_sensitivity numeric,
       review_propensity numeric, review_negativity_bias numeric,
       return_prob_base numeric, weather_sensitivity jsonb)

customer(id uuid pk, world_id fk, persona_id fk, is_synthetic bool, pseudonym text,  -- no PII
        first_seen_at, last_seen_at, visits int, lifetime_spend numeric(12,2),
        satisfaction_ema numeric, churned bool, loyalty_tier text)

party(id uuid pk, world_id fk, customer_id fk, size int, channel_id fk, arrived_at,
      queued_at, ordered_at, seated_at, left_at, table_id fk null,
      outcome text check (outcome in ('served', 'balked', 'reneged', 'cancelled')),
      balk_reason text, patience_s numeric)
```

ops (orders, cooking, delivery)

```
"order"(id uuid pk, world_id fk, channel_id fk, party_id fk null, customer_id fk null,
       external_ref text, order_no int,
       status text check (status in ('placed', 'accepted', 'rejected', 'in_prep', 'ready', 'handed_off',
                                     'served', 'completed', 'cancelled', 'refunded')),
       placed_at, accepted_at, promised_at, first_task_at, ready_at, handed_off_at, completed_at,
       subtotal numeric(10,2), discount numeric(10,2), tax numeric(10,2), commission numeric(10,2),
       packaging_cost numeric(10,2), total numeric(10,2), cogs numeric(10,2), contribution numeric(10,2),
       payment_method text, priority_score numeric, acceptance_decision_id fk null,
       is_batched bool, remake_count int, notes text)

order_item(id uuid pk, order_id fk, menu_item_id fk, qty int, unit_price numeric(10,2),
          modifiers_snapshot jsonb, special_instructions text,
          status text, batch_id fk null, from_prep bool,
          started_at, ready_at, served_at, quality_at_handoff numeric(4,3),
          temperature_ok bool, cogs numeric(10,2), co2e_g numeric)

order_item_modifier(order_item_id fk, modifier_option_id fk, price_delta numeric(8,2), primary key(order_item_id,
                                                    modifier_option_id))

kitchen_task(id uuid pk, world_id fk, order_item_id fk null, prep_batch_id fk null, recipe_step_id fk,
            station_id fk, equipment_id fk null, staff_id fk null, cook_batch_id fk null,
            status text, queued_at, planned_start, started_at, ended_at, duration_s numeric, was_rework bool)

cook_batch(id uuid pk, world_id fk, station_id fk, equipment_id fk, size int,
          created_at, held_until, started_at, ended_at, channels int[], decision_id fk null,
          time_saved_s numeric)                -- vs cooking items separately

delivery_handoff(id uuid pk, order_id fk unique, aggregator text, rider_eta_pred_at, rider_eta_p90_at,
                rider_assigned_at, rider_arrived_at, picked_up_at, shelf_slot int,
                food_wait_s numeric,  -- ready → pickup (quality loss)
                rider_wait_s numeric) -- arrived → pickup (aggregator penalty)

receipt(id uuid pk, order_id fk, receipt_no text, printed_at, lines jsonb, tax_breakdown jsonb, qr_payload text)
payment(id uuid pk, order_id fk, method text, amount numeric, gateway_fee numeric, ts, status)

review(id uuid pk, world_id fk, order_id fk, customer_id fk, channel_id fk, rating smallint,
      text text, sentiment numeric, causes jsonb,  -- {"wait":0.6,"cold_food":0.3,"price":0.1}
      created_at, responded bool)
```

fin

```
ledger_entry(id uuid pk, world_id fk, ts, account text,  -- revenue, discount, cogs, commission, packaging, labour,
              energy,
              -- waste, rent, maintenance, capex, depreciation, refund,
              donation_writeoff
              amount numeric(12,2), ref_type text, ref_id uuid, memo text)
```

```

investment_catalog(id pk, kind text, -- table, chairs, equipment, storage, staff_hire, shift_extension, marketing,
    training
    name, sku, capex numeric(12,2), opex_per_day numeric(10,2), lead_time_h int,
    effect jsonb, -- {"add_table":{"seats":2}} | {"add_equipment":"espresso_2grp"}
    sprite text, footprint_m2 numeric)

investment(id uuid pk, world_id fk, catalog_id fk, decided_at, decided_by text, advisor_rec_id fk null,
    expected_delta_profit_day numeric, expected_payback_days numeric, ci jsonb,
    realised_delta_profit_day numeric, status text)

daily_kpi(world_id fk, date date, revenue numeric, gross_profit numeric, net_profit numeric,
    orders int, orders_by_channel jsonb, avg_wait_s jsonb, p95_wait_s jsonb, sla_breach_rate jsonb,
    balks int, reneges int, rating_avg numeric, reviews_neg int, table_turns numeric,
    labour_hours numeric, revenue_per_labour_hour numeric, food_cost_pct numeric,
    waste_kg numeric, waste_value numeric, donated_kg numeric, energy_kwh numeric, co2e_kg numeric,
    staff_overload_minutes numeric, price_changes int, primary key (world_id, date))

```

m1 (forecasts, policies, decisions, analysis)

```

external_signal(id pk, cafe_id fk, ts, kind text, -- temp_c, rain_mm, aqi, holiday, festival, cricket_match,
    exam_week, payday
    value numeric, label text, source text)

model_registry(id uuid pk, kind text, -- demand_forecast, elasticity, prep_time, rider_eta, rl_policy, adversary
    name, version text, framework text, artifact_uri text, params jsonb, metrics jsonb,
    trained_on jsonb, git_sha text, trained_at, is_champion bool)

forecast_run(id uuid pk, model_id fk, world_id fk, issued_at, horizon_min int, granularity_min int)
forecast(run_id fk, target text, key text, bucket_start timestamptz, mean numeric,
    p10 numeric, p50 numeric, p90 numeric, actual numeric null, primary key(run_id, target, key, bucket_start))

policy(id pk, code char(1), -- A B C D E
    name, kind text, model_id fk null, config jsonb, description)

policy_decision(id uuid pk, world_id fk, policy_id fk, ts,
    decision_type text, -- accept_order, throttle_channel, start_prep, prep_qty, price_change, feature_item,
    -- hide_item, strategy_switch, batch_hold, dispatch, reorder, staff_reassign
    target_ref jsonb, action jsonb, alternatives jsonb, value_estimate numeric,
    explanation text, top_factors jsonb, obs_ref text) -- pointer to Parquet observation row

training_run(id uuid pk, policy_id fk, algo text, phase text, -- bc, curriculum_1d, curriculum_7d, curriculum_28d,
    adversarial
    hyperparams jsonb, env_steps bigint, wall_time_s numeric, curves_uri text,
    best_eval jsonb, started_at, finished_at)

evaluation(id uuid pk, policy_id fk, scenario_id fk, seed bigint, days int, world_id fk,
    profit numeric, kpis jsonb, created_at)

disruption(id uuid pk, world_id fk, kind text, -- staff_absent, staff_late, equipment_down, supplier_delay,
    supplier_short,
    -- rider_shortage, power_cut, demand_spike, price_shock,
    platform_outage
    target_ref jsonb, severity numeric, starts_at, ends_at, source text check (source in
    ('scenario','adversary','manual')))

bottleneck_snapshot(id uuid pk, world_id fk, ts, window_min int, resource_type text, resource_ref text,
    utilisation numeric, queue_len_avg numeric, wait_attrib_s numeric, active_period_share numeric,
    shadow_price numeric, is_primary bool)

counterfactual(id uuid pk, base_world_id fk, fork_ts, change jsonb, seeds int, horizon_days int,
    delta_profit_mean numeric, delta_profit_ci jsonb, delta_kpis jsonb, payback_days numeric, created_at)

sim_event(world_id fk, seq bigint, ts timestamptz, type text, entity_ref text, payload jsonb,
    primary key(world_id, seq)) -- only for live/demo worlds; train/eval go to Parquet

```

7.4 Indexes, integrity and invariants

- Indexes: `order(world_id, placed_at)`, `order(world_id, status)`, `order_item(order_id)`, `kitchen_task(world_id, status, station_id)`, `stock_lot(world_id, ingredient_id, expires_at)` where `status` in `( 'sealed','opened' )`, `price_history(world_id, menu_item_id, effective_from desc)`, `policy_decision(world_id, ts)`, and a BRIN index on `sim_event(ts)`.
- Invariants, tested with Hypothesis property tests:
 - Inventory is conserved: $\Sigma \text{ movements} = \Delta \text{ stock}$.

- No negative lots.
- An order's total equals its lines + tax – discount.
- The ledger balances to the KPIs.
- A staff member can't be in two tasks with attention summing to more than 1.
- Equipment slots are never over capacity.
- **Snapshots:** a price is copied onto `order_item.unit_price`, and modifiers onto `modifiers_snapshot`, so history never changes when the menu changes.
- **Privacy:** no real PII. Customers are pseudonymous. In production, any phone or email would be stored hashed in a separate restricted schema.

7.5 Telemetry in Parquet (`data/runs/{world_id}/`)

- `events.parquet` : every sim event
- `staff_state.parquet`, `equipment_state.parquet`, `queue_state.parquet` : per-minute
- `trajectories/*.parquet` : obs, action, reward, done, info for RL
- `arena/{eval_id}.parquet` : per-day KPIs

These are queried with DuckDB in notebooks and by the `/api/analytics` endpoints.

8. The simulator (digital twin)

8.1 Engine

- **Custom discrete-event engine.** A binary heap of `(time, priority, seq, event)` with pure-Python handlers, optimised with `__slots__` dataclasses. We chose this over SimPy for speed, deterministic ordering, easy **pickle/fork** and fine control over decision points.
- **RNG streams:** one `numpy.random.Generator` per subsystem (arrivals, choice, durations, failures, riders, reviews), derived from a world seed via `SeedSequence.spawn`. This makes **common random numbers** possible across policies: the same customers arrive with the same tastes regardless of policy.
- **Performance target: 1 simulated day (about 300–600 orders) in under 0.3 s headless.** Training runs with `SubprocVecEnv` across all cores.
- **Decision points**, where the engine yields to the policy:
 - `MANAGER_TICK` every 15 sim-minutes (pricing, pre-prep, strategy, throttles)
 - `ORDER_ARRIVED`, especially aggregator orders, for accept / delay / reject
 - `RESOURCE_FREE` : a staff member or equipment slot frees up (dispatch / batch hold)
 - `DAY_START` / `DAY_END` (roster, purchase orders, investments)

8.2 Demand generation

1. **Arrivals:** a non-homogeneous Poisson process per persona × channel: $\lambda(t) = \text{base} \times \text{dow_profile} \times \text{slot_profile}(t) \times \text{weather_mult} \times \text{event_mult} \times \text{reputation_mult} \times \text{price_index_mult} \times \text{trend} \times \text{noise}(\text{lognormal})$
2. **Groups:** party size sampled from each persona's distribution.
3. **Menu choice:** a **multinomial logit** over the visible menu: $U_{ij} = \alpha_j + \beta_{\text{persona}} \cdot \ln(\text{price}_j / \text{ref_price}_j) + \gamma \cdot \text{featured}_j + \delta \cdot \text{weather_fit}_j + \epsilon$ with an outside option (walk away). This gives realistic **price elasticity**, substitution between items (a pricier latte pushes some people to cappuccino), and add-ons.
4. **Reference-price fairness.** Customers remember past prices, and a price above the reference carries a loss-aversion penalty (prospect-theory style, $\lambda \approx 2$). This *naturally punishes* aggressive surge pricing.
5. **Balking:** customers who see a queue that is too long or no free table may leave on arrival. This depends on the persona.
6. **Reneging:** customers walk out when their patience clock runs out (offline only). Delivery customers cancel if the order isn't accepted in time.
7. **Dwell:** seated time follows a lognormal by persona. Laptop campers stay long and order refills.

8.3 Kitchen model

- Each order item expands its recipe into a **task DAG** (`recipe_step` + modifier deltas).
- Tasks need a **station + equipment slot + staff member with the skill** (staff attention can be partial: one barista can watch the oven while pulling shots).
- **Durations** are $\text{lognormal}(\mu, \sigma) \times \text{staff speed} \times (1 + \text{fatigue penalty}) \times \text{learning curve}$.
- **Batching**: compatible tasks on a batchable step merge, e.g. 4 paninis in one press, a 1-litre steamed milk pitcher for 3 lattes, or a fryer basket. Time is $t \cdot (1 + f \cdot (n-1))$.
- **Quality decay**: after `ready_at` , quality follows $q(t) = 2^{-(t/\text{half_life})}$. Quality at handoff affects satisfaction and review probability. *This is why delivery orders should be finished just in time for the rider, not early.*
- **Errors and remakes**: errors happen with probability depending on staff error rate $\times$ load. A remake consumes extra inventory and time.
- **Dish pit**: a finite cup and plate pool, so dishwashing can become the bottleneck.

8.4 Delivery and aggregators

- An aggregator order arrives and must be accepted within `acceptance_timeout_s` . A **promised prep time** is declared at acceptance.
- **Rider arrival** comes from a learned or forecast ETA distribution and is affected by rain, peak hours and rider shortage disruptions.
- **Penalties**: rider waiting (the platform's prep-time score) and food waiting (quality decay). Late handoff lowers the platform rating, which feeds the platform's ranking multiplier and therefore future delivery demand.
- **Economics**: a commission % (assumption: about 18–30% range), packaging cost, and a payment gateway fee. Platform listing price can differ from dine-in, within a configurable parity limit.
- **Throttle actions**: *open, inflate prep time +5/+10 min, or pause the store for N minutes*. Pausing is realistic and has a cost: lost orders, and a ranking hit if used too often.

8.5 Reputation and long-term effects

- Each completed order produces satisfaction: $S = w1 \cdot (1 - \text{lateness}/\text{patience}) + w2 \cdot \text{quality} + w3 \cdot (\text{value_for_money}) + w4 \cdot \text{accuracy} - w5 \cdot \text{price_unfairness}$
- Review probability depends on persona, extremeness of S and channel. Ratings form a running **reputation** per channel (a Bayesian average).
- **Reputation multiplies future demand** (`reputation_mult`). Customers who leave unsatisfied have a lower return probability. So a policy that squeezes today pays for it next week. This is what makes **multi-day episodes** essential and lets RL learn long-term policies.

8.6 Real-world factors modelled

Factor	Model
Day of week, time of day	Persona slot profiles (commuter 7:30–9:30, students 15:00–18:00, leisurely weekends)
Weather	Hot → cold drinks ↑; rain → dine-in ↓, delivery ↑, rider ETA ↑
Calendar	Holidays, festivals (sweet items ↑), exam weeks (students camp ↑), paydays (spend ↑), cricket matches (delivery spike)
Trends	Item lifecycle, new launches, menu fatigue
Ingredient prices	Random-walk shocks (milk, coffee, avocado), seasonal produce
Supplier	Lead time variance, partial fills, MOQ, delivery days, cut-offs
Staff	Shifts, lateness, absence, fatigue, breaks, skill mix, learning curve, wages, overtime
Equipment	Warm-up, slots, MTBF/MTTR failures, degradation, energy draw
Space	Tables, seats, combinable tables, cleaning time, outdoor seating vs rain

Factor	Model
Channels	Commission, packaging, acceptance timeouts, rider ETA, platform rating effects
Tax	GST on bill (configurable, 5% default for restaurants)
Utilities	Energy kWh × tariff (time-of-day tariff optional), power cuts
Fixed costs	Rent and maintenance per day, so profit is real profit and not just contribution

8.7 Synthetic history generation

`brew-sim --days 120 --policy A --scenario mixed` produces realistic "past data" (orders, items, timings, weather, events) for training forecasters and elasticity models. This mirrors what a real cafe would upload from its POS. We randomise persona mixes, menu prices (with exploration noise so elasticity is identifiable) and calendars.

8.8 Calibration (realism story for judges)

- Default parameters come from public, generic cafe operations knowledge (prep times, dwell times, typical food cost ratios), all stored in YAML and fully editable.
- Calibration hook:** upload a real POS CSV and fit arrival profiles, item mix and prep-time distributions (maximum likelihood / Optuna). The twin then mirrors *your* cafe. In the demo we show this with a sample CSV.

9. Customer personas

Persona	Arrives	Group	Channel	Price sens.	Patience	Dwell	Basket	Review behaviour
Commuter	7:30–9:45, 17:30–19:00 weekdays	1	takeaway 70%, dine-in 30%	low–med	very low (≈4 min)	0–10 min	espresso drinks, croissant	rarely reviews, but churns silently if slow
Student	14:00–19:00, exam weeks ↑	1–4	dine-in 70%, delivery 30%	high	medium	60–150 min	cold coffee, fries, maggi/pasta, combos	chatty, reviews often; loves deals
Leisurely	weekends 10:00–16:00	2–3	dine-in	low	high	45–90 min	specialty drinks, desserts, brunch	reviews ambience and quality
Remote worker	10:00–17:00 weekdays	1	dine-in	medium	medium	120–240 min	refills, sandwiches	sensitive to table availability
Family	weekends, evenings	3–5	dine-in	medium	low–med (kids)	45–75 min	shakes, waffles, mains	reviews wait time
Office bulk	11:00–12:30, 16:00	1 order, 8–20 items	takeaway / delivery	low	deadline-hard	0	coffees en masse	very sensitive to lateness
Delivery-at-home	12:00–15:00, 19:00–23:00, rain ↑	1–3	Zomato / Swiggy	medium	medium (platform ETA)	0	mains, combos, desserts	rates food temperature and packaging

Persona	Arrives	Group	Channel	Price sens.	Patience	Dwell	Basket	Review behaviour
Tourist	random, weekends	2–4	dine-in	low	medium	30–60 min	signature items (Rose Latte)	review-happy, high-impact
Regular	personal habit times	1	any	low	med-high	habit	"the usual"	loyal; churns after 2 bad visits

Offline-critical rule: offline personas (commuter, family, office pickup, walk-ins) have **higher review propensity and negativity bias for waiting**. The agent's acceptance and priority logic accounts for this through persona-weighted lateness penalties.

10. Inventory model

- Units and conversions.** Each ingredient has a base unit (g / ml / pc). We convert pack ↔ base (a 1 L carton = 1000 ml; a bag of beans = 1000 g), and density handles g↔ml.
- Lots and batches.** Every receipt creates a lot with cost, received date, expiry and quality. Consumption picks **FEFO** (first-expiry-first-out) by default.
- Open vs sealed.** Opening a lot moves its expiry forward to `min(expires_at, opened_at + shelf_life_opened)` . Opened milk lasts days, and opened cream less.
- Quality curves.** Produce (avocado, berries) degrades and quality affects dish quality. Overripe produce goes to markdown or to waste.
- Storage capacity and zones.** Chilled, frozen and ambient zones have volume limits. A bigger fridge is a real investment that unlocks larger, cheaper orders.
- Prep items (work in progress).** Cold brew concentrate (12 h lead), croissant dough (proofing), sauces, chopped veg and brewed chai base each have **hold times** and **quality half-lives**. Pre-prep decisions create these.
- Yield and waste factors.** We model trim loss, spillage and errors.
- Packaging is inventory:** cups, lids, kraft bags, sleeves and straws. Running out of bags blocks delivery orders.
- Suppliers.** Each has lead time distributions, MOQs, delivery days, cut-offs, fill-rate uncertainty and price shocks.
- Replenishment policies:**
 - Baseline: **(s, S)** with forecast-driven reorder points
 - Optimised: **perishable newsvendor** with lead time (minimises expected overage + underage + expiry)
 - RL adds signals on top, e.g. "order extra berries before the festival"
- Stock-outs propagate.** When an item is out, the menu auto-86s it, substitutions are allowed if the recipe permits, and the customer choice model re-routes demand.
- Shrinkage** (random small losses) gets reconciled via cycle counts.
- Waste taxonomy:** expiry, spoilage, error/remake, overproduction (pre-prep not sold), plate waste (optional). Each type is costed and converted to CO₂e.
- Donation flow.** At close, safe surplus items (bakery, sealed) go to a donation crate. These are logged as `donate` movements, written off for accounting and counted for social impact.

11. Forecasting models

11.1 Targets

Target	Granularity	Horizon	Used by
Item demand by channel	15 min	0–6 h (intraday), 1–7 days	pre-prep, pricing, staffing
Order arrivals by channel and persona	15 min	0–6 h	acceptance, throttle

Target	Granularity	Horizon	Used by
Ingredient consumption	daily	1–14 days	purchasing
Prep task durations	per task	n/a	scheduler, promise times
Rider ETA	per order	n/a	JIT delivery cooking
Table occupancy / turnover	15 min	0–4 h	seating, balk risk

11.2 Models

- **Baselines:** seasonal naive, ETS / AutoARIMA (statsforecast). These are the floor we must beat.
- **Main model:** a global LightGBM with quantile objectives (P10/P50/P90) across all item-channel series. Features:
 - Calendar: dow, slot, holiday, festival, exam week, payday
 - Weather: temperature, rain, forecast
 - Lags and rolling stats: same slot yesterday or last week, rolling 7-day mean
 - Prices: current price / reference price, featured flag
 - Context: reputation, aggregator open/paused, recent trend, persona-mix priors
- **Intraday nowcasting:** a Bayesian update of today's level from realised orders so far (a Gamma–Poisson update on the multiplier). This makes forecasts react within the morning rush.
- **Hierarchical reconciliation** (item → category → total, MinT or simple bottom-up) so the numbers add up.
- **Prep time model:** a quantile GBM on (recipe step, staff, fatigue, batch size, load).
- **Rider ETA model:** a quantile GBM on (hour, rain, platform, distance proxy, recent ETAs).
- **Price elasticity:** a hierarchical Bayesian log-log demand model per item, pooled by category. We fit it on synthetic history with price exploration, and it outputs a posterior β for the optimiser. The fallback is constrained Poisson GLM regularised to category means.

11.3 Evaluation

- Rolling-origin backtests. Metrics: **WAPE**, **pinball loss** per quantile, coverage of P10–P90, and bias.
- Results are logged to model_registry.metrics . The UI shows a "forecast vs actual" overlay.

11.4 Why probabilistic matters

Pre-prep is a **newsvendor problem**. The optimal quantity is the quantile $q^* = F^{-1}(c_u / (c_u + c_o))$, where c_u is the cost of being short (wait-time penalty + lost sale) and c_o is the cost of excess (waste + quality decay). The RL agent controls "aggressiveness" = which quantile to prep to, conditioned on state.

12. Optimisation models

Model	Tool	Decision	Role
Kitchen scheduler	OR-Tools CP-SAT	Assign tasks → staff/equipment/time, form batches; minimise $\sum w_i \cdot \text{lateness}_i$ + quality loss – batching savings	Policy C core; RL's low-level executor
Order acceptance / promise	CP-SAT feasibility + slack check	Can we accept this aggregator order and still meet existing promises? What promise time?	C and D
Pre-prep quantities	Newsvendor with hold-time constraint	How much of each prep item to make and when (latest start = need time – lead time)	B uses P50, C uses cost-ratio quantile, D learns κ

Model	Tool	Decision	Role
Menu pricing	MILP over a discrete price ladder	Pick prices per item for the next window to maximise $\sum (p - c) \cdot d(p)$ subject to kitchen capacity per station, inventory, table capacity, and fairness caps (	Δp
Capacity LP (relaxation)	HiGHS via OR-Tools	Fluid model of the next 4 h: maximise contribution subject to station-minutes, staff-minutes, seats $\times$ time, inventory	Shadow prices = value of one more unit of each resource (bottleneck and investment)
Replenishment	Perishable newsvendor / (s,S) search	Purchase orders	All policies (configurable)
Roster	CP-SAT	Shifts for tomorrow vs forecast, labour law constraints, fairness	Day-start decision

Rolling horizon: the scheduler re-solves on every event with a time limit (around 50–200 ms) and warm-starts from the previous solution. This keeps real-time play smooth at 10 $\times$.

13. Reinforcement learning design

13.1 What RL decides

The scope is exactly what was asked for:

RL responsibility	Concrete action	Frequency
When to start preparation	For each prep item: start a batch now / wait	manager tick (15 min) + event triggers
How aggressively to pre-prep	$\kappa \in \{\text{off, P50, P65, P80, P90}\}$ per prep-item class	manager tick
Dynamic menu intervention	Per category: price step $\{-10, -5, 0, +5, +10\}\%$; feature one item; 86 up to k slow items under load; trigger near-expiry happy hour	manager tick
Choosing between operational strategies	Strategy $\in \{\text{FCFS, EDF (earliest deadline), dine-in-first, delivery-JIT, batch-max, throughput-max}\}$ feeding the scheduler's weights	manager tick
Order intake across channels	Per aggregator: open / +5 min / +10 min / pause; accept or reject each incoming order	event + tick
Batch hold	When a slot frees: start now vs hold up to $\{30, 60, 120\}$ s} to batch	resource-free event
Long-term kitchen policies	Learned implicitly across 7–28-day episodes with reputation, loyalty, supplier ordering and investment states carried over	episode

13.2 Architecture: hierarchical, hybrid

Manager policy π_M (PP0/RecurrentPP0, every 15 sim-min)
→ prices, features, 86s, κ pre-prep, strategy weights, channel throttles, batch window
Dispatcher (event-level)
→ D-lite: CP-SAT scheduler parametrised by π_M 's strategy weights + batch window
→ D-full (stretch): learned dispatch policy π_D (MaskablePP0) choosing next task/batch-hold
Acceptance head (event-level, small MLP or rule on π_M 's throttle + feasibility check)
Safety shield: hard constraints (price caps, inventory feasibility, staff limits) mask invalid actions

Why hybrid: pure end-to-end RL on per-second kitchen dispatch is sample-hungry and risky in a hackathon. RL controlling *what matters strategically*, with the solver executing, gives us a working **Policy D early** (D-lite). We then push further with D-full if

time permits. Both are genuine RL, and both are "learned restaurants".

13.3 Observation (manager)

- **Time:** sin/cos of time of day, dow one-hot, day index in episode, holiday/event flags, weather now and forecast.
- **Demand:** forecast P10/P50/P90 for the next 4×15 -min slots per category and channel; realised-vs-forecast ratio today (nowcast).
- **Queues:** open orders per channel; slack-to-promise histogram; persona mix of waiting customers; customers waiting with patience below 30%.
- **Resources:** per-station utilisation (last 15 min), queue length, equipment status (ok/degraded/down), staff present, skills coverage, fatigue mean/max, tables free / occupied / dirty.
- **Inventory:** per key ingredient and prep item, days of cover, stock in expiry buckets (<4 h, <24 h, >24 h), incoming POs.
- **Economics:** current price index per category, today's profit so far, cash, channel reputations, recent price-change count.
- **Disruption flags:** what is currently broken or absent.
- About **150–250 floats**, normalised (running mean/std via `VecNormalize`).

13.4 Action space

`MultiDiscrete` with **action masking** (`sb3-contrib MaskablePPO`): - 5 categories $\times$ 5 price steps - 6 prep classes $\times$ 5 κ levels - 6 strategies - 2 aggregators $\times$ 4 throttle levels - 4 batch windows - top-3 86 toggles

Masks enforce: price within [min, max]; no change if one was made in the last 2 h; can't 86 an item that's already in an open order; can't prep without inventory.

13.5 Reward

Per manager step (15 min):

```
r_t = Δprofit_t                                     # revenue - COGS - commission - packaging - labour - energy - waste -
fixed share
- λ_late · Σ_i w_persona(i) · late_i               # persona-weighted lateness (offline weighted higher)
- λ_walk · walkouts_t · E[LTV_persona]             # lost lifetime value
- λ_rev · Σ ΔE[reputation] · demand_elasticity_to_rating # shaping; reputation is also endogenous
- λ_churn · |Δprice|_count                          # price stability / fairness
- λ_waste · waste_kg_t · (₹ + CO2e shadow cost)
- λ_staff · overload_minutes_t                     # staff wellbeing (sustained >95% utilisation)
terminal: + V_inventory (salvage value of usable stock) - V_obligations (open orders)
```

- All λ are in ₹ units, so the reward is "profit with priced externalities". Shaping terms are **annealed** toward pure profit plus true long-run effects. The sim already makes reputation endogenous, so in the final phase the agent optimises *true* multi-day profit.
- The **evaluation metric is always the true P&L** (net profit), plus the separately reported KPIs. That keeps comparisons fair.

13.6 Training pipeline

1. **Synthetic history** (§8.7) → train forecasters and elasticity.
2. **Behaviour cloning warm start** from Policy C's decisions over 200 simulated days. The manager learns to imitate the solver-based pricing and pre-prep. This gives a strong start and cuts PPO time sharply.
3. **PPO curriculum:**
 - Stage 1: single-day episodes, calm scenarios, shaping on
 - Stage 2: 7-day episodes, mixed scenarios (weather, events), domain randomisation of persona mix, prices, staff speed ($\pm 20\%$)
 - Stage 3: 28-day episodes with reputation, loyalty and purchase-order dynamics; shaping annealed
4. **Adversarial phase** (§14).
5. **Evaluation** on held-out seeds and scenarios → registry → export to ONNX for fast CPU inference in the API.

Hyperparameter defaults: `n_envs=16`, `n_steps=512`, `batch=4096`, `gamma=0.995` (15-min steps; long horizon), `gae_lambda=0.95`, `lr=3e-4` → `1e-4` (linear), `ent_coef=0.01`, `clip=0.2`, `network [256, 256]` (or LSTM 128 for RecurrentPPO). Optuna sweeps on the 7-day stage if time allows.

Compute budget estimate: 28-day episode $\approx 28 \times 56$ manager steps ≈ 1.6 k steps. 5 M steps ≈ 3 k episodes ≈ 85 k sim-days. At 0.3 s/day across 16 cores that is about 25–30 minutes of wall time. **Feasible on a laptop in a hackathon.**

13.7 Explainability

- Each decision stores **top factors**: gradient $\times$ input or a SHAP KernelExplainer on a distilled surrogate tree (a small decision tree fitted to the policy's actions, which is also a nice "policy summary" slide).
- Natural-language notes** come from templates filled with the factor values, e.g. *"Raised iced latte +₹10: kitchen 91% busy, P90 cold-drink demand +40% (34°C), stock fine."* Optional LLM paraphrase via "Ask Brew" (§19).
- Counterfactual button**: "what if the agent hadn't done this?" forks the world and replays 30 minutes without the action, then shows the Δ .

14. Adversarial training ("chaos phase")

Goal: a policy that degrades gracefully. We judge it by the tail (CVaR) as well as the average.

- Disruption library** (§7 `disruption.kind`): staff absent or late, equipment down (with MTTR), supplier late or short, rider shortage (ETA $\times 2$), power cut, demand spike (bus of tourists, cricket final), price shock (milk +20%), platform outage.
- Adversary agent (RARL, robust adversarial RL)**: a second PPO policy π_A with a **disruption budget** per episode (e.g. 3 "chaos points"). It chooses *which* disruption to trigger and *when*, and is rewarded with $-\text{profit}$ of the protagonist. We alternate training: k iterations of π_M against a frozen π_A , then k iterations of π_A against a frozen π_M . A budget and plausibility constraints (e.g. at most 1 staff absence per day, failures weighted by MTBF) keep it realistic and stop it from being unbeatable.
- Domain randomisation mix**: 50% random disruptions, 30% adversary, 20% calm. This prevents overfitting to the adversary.
- Learned adaptive behaviours we expect, and will show**:
 - Pre-prep $\uparrow$ before a known shortage
 - Shifting menu weight away from the broken station (86 paninis when the press is down; feature cold items)
 - Throttling aggregators early during staff absence
 - Re-routing skills
 - Ordering safety stock when supplier reliability drops
- Metrics**: mean profit, **CVaR₁₀%** of daily profit, SLA breach under disruption, and recovery time (minutes to return to normal wait times after a shock).

15. Bottleneck detection and investment advisor

"What is currently limiting throughput?" is answered with three complementary methods, each shown in the UI:

- Utilisation and queueing (fast, live)**. For each resource (each station, each equipment type, each staff role, tables, dish pit, cups, cash counter, specific ingredients) we compute utilisation ρ , queue length and **wait-time attribution**: how many seconds of order lateness were spent waiting for that resource. We use Little's Law ($L = \lambda W$) to sanity-check.
- Active-period method (Roser)**. The resource that is active (busy without interruption) for the longest share of time is the momentary bottleneck. Tracking it over time reveals **shifting bottlenecks**, e.g. the espresso machine at 8 a.m., tables at 1 p.m., and riders in the 8 p.m. rain.
- Shadow prices (economic)**. The capacity LP (§12) gives the dual value of each constraint: **₹ of extra profit per extra unit-hour of that resource**. This turns "busy" into "worth money".

Investment advisor (counterfactual simulation). - For each catalogue item (table +2 seats, second espresso machine, bigger fridge, morning barista, dishwasher upgrade, extra 20 L oat milk standing order, and so on), it **forks the current world** and simulates the next 7–14 days $\times$ 20 seeds with and without the change, under the current policy. - It reports **Δ profit/day (mean, 90% CI)**, **payback days**, **Δ wait**, **Δ waste**, **Δ CO₂e**, and ranks by **ROI = Δ profit / cost** with uncertainty. - Shadow prices pre-screen candidates so only promising ones get simulated (keeps it fast). - **Profit is spendable**. Cash earned in the sim can buy items in the shop. The item appears (table animation, machine delivered), and the next day's arena shows the effect. This closes the loop: **profit $\rightarrow$ investment $\rightarrow$ more profit**. - UI copy: *"Your #1 constraint this week: espresso group heads 7:45–9:30 ($\rho=0.97$). A 2nd machine: +₹2,140/day (CI ₹1,600–2,700), payback 41 days. Tables are not your constraint ($\rho=0.61$), so don't buy tables."*

16. Policy Arena and evaluation

16.1 Policies

Code	Name	Intake	Dispatch	Pre-prep	Pricing	Purchasing
A	Naive restaurant	accept all	FCFS per station	none (cook to order)	static	fixed weekly
B	Heuristic restaurant	accept all; pause aggregator if queue > N	FCFS + batch similar items within a 60 s window; dine-in priority	fixed P50 morning prep	static + happy hour in dead hours	(s,S)
C	Optimised restaurant	CP-SAT feasibility accept	CP-SAT rolling-horizon scheduler	newsvendor quantile	MILP pricing every hour	perishable newsvendor
D	Learned restaurant	RL throttle + feasibility	CP-SAT parametrised by RL (D-lite) / learned dispatcher (D-full)	RL κ + timing	RL interventions (shielded)	newsvendor + RL signals
E	Oracle (upper bound)	perfect knowledge of future arrivals	offline optimum over the day (CP-SAT with long time limit)	perfect	MILP with true demand	perfect

E is not achievable in reality. It shows how much headroom remains: "D captures X% of the oracle gap from A".

16.2 Protocol

- **Common random numbers:** identical seeds yield identical customer streams and disruption schedules across policies. Differences come from policy, not luck.
- **Scenarios:** weekday_normal , weekend_brunch , rainy_delivery_surge , festival , exam_week , heatwave , chaos_random , chaos_adversary (adversary frozen from training, plus a **held-out** adversary not seen in training).
- **Scale:** 30 seeds × 14 days per scenario per policy. Report the mean and **paired bootstrap 95% CI** of the difference vs A. We also run a Wilcoxon signed-rank test.
- **KPIs:**
 - Economics: net profit, revenue, contribution margin, food cost %
 - Service: avg and P95 wait by channel, SLA breach %, walkouts, rating
 - Efficiency: table turns, revenue per labour hour, batch rate
 - Environment and people: waste kg / ₹, CO₂e, staff overload minutes
 - Pricing: price changes per day
- **Ablations** (to show each component matters):
 - D without forecasts
 - D without batching
 - D without pricing
 - D without adversarial training (tested on chaos)
 - D with a 1-day horizon vs 28-day (shows long-term learning: reputation)

16.3 Presentation

- A live 2×2 race plus a cumulative profit chart.
- A results table with CIs, plus a radar chart (profit, speed, rating, waste, staff wellbeing) per policy.

- A "Where the money came from" waterfall (A → D): + batching time savings, + fewer walkouts, + pricing, – waste avoided, + delivery JIT, and so on. This is computed by ablation deltas.

17. API and real-time protocol

17.1 REST (FastAPI, OpenAPI auto-docs at /docs)

```
GET /api/cafe
GET /api/worlds/{id}
POST /api/worlds/{id}/fork
GET /api/worlds/{id}/state
GET /api/menu?world=
GET /api/orders?world=&status=
GET /api/staff?world=
GET /api/forecast?world=&target=&horizon=
GET /api/bottlenecks?world=
GET /api/decisions?world=&since=
POST /api/arena {policies, scenario, seeds, days}
GET /api/kpis?world=&from=&to=
POST /api/calibrate (CSV upload)

POST /api/worlds {kind, scenario, policy, seed, speed}
POST /api/worlds/{id}/control {play|pause|speed|seek}
POST /api/worlds/{id}/chaos {kind, target, severity}
# full snapshot for (re)hydration
# live prices, features, 86s
GET /api/inventory?world=
GET /api/inventory/{ingredient}/lots
GET /api/equipment?world=
GET /api/advisor?world=
POST /api/invest {catalog_id}
GET /api/decisions/{id}/explain
GET /api/arena/{id}
GET /api/impact?world=
POST /api/ask {question} # Ask Brew
```

17.2 WebSocket /ws/worlds/{id}

Server → client messages (orjson, batched every 50–100 ms of wall time):

```
{
  "seq": 10432,
  "t": "2026-10-03T08:14:22+05:30",
  "type": "order.placed",
  "data": {
    "order_id": "...",
    "no": 57,
    "channel": "zomato",
    "items": [
      {
        "sku": "LAT-ICED",
        "mods": ["oat", "extra_shot"]
      }
    ],
    "promised_at": "...",
    "persona": "delivery_home"
  }
}
```

Event types include:

- Customers: `customer.arrived/queued/ordered/seated/left/balked/renege`
- Orders: `order.placed/accepted/rejected/ready/handed_off/served/cancelled`
- Kitchen: `task.started/finished, batch.formed/started, prep.started/ready/expired`
- Pricing and menu: `price.changed, menu.featured/hidden`
- Inventory: `lot.opened/expired/received, po.created, stock.low`
- Staff: `staff.arrived/break/absent`
- Equipment: `equipment.down/up`
- Delivery: `rider.arrived/picked_up`
- Money and reviews: `receipt.printed, review.posted`
- Agent and analysis: `decision.made, bottleneck.changed, kpi.tick, chaos.triggered, investment.delivered, day.ended`

The client **store** applies events to its state (event sourcing). Components subscribe to slices, and the AudioDirector subscribes to event types. On reconnect, the client fetches `/state` and resumes from `seq`.

18. Event → Visual → Sound map (the "juice" table)

Event	Visual	Sound
<code>customer.arrived</code>	Door swings, customer walks in	door chime + footsteps
<code>customer.ordered</code> (dine-in)	Speech bubble with item icons → ticket prints	ticket printer chirp
<code>order.placed</code> (aggregator)	Tablet on counter lights up, ticket slides onto rail with a ZOMATO/SWIGGY-coloured stamp	generic delivery chime
<code>order.rejected</code> /throttle	Ticket stamped "PAUSED" in red, slides off	stamp thump
<code>batch.formed</code>	Tickets slide together, pink paperclip snaps, "x3" badge	clip click
<code>task.started</code>	Staff walks to station, station animates	station-specific loop
<code>prep.started</code>	Mise-en-place container fills	chop / drip
<code>prep.expired</code>	Container greys, slides into bin	soft "bonk" + bin lid

Event	Visual	Sound
order.ready	Plate on the pass / bag on the shelf, flap board flips to READY	pass bell / flap clatter
rider.picked_up	Rider takes bag, bag fades	bag crinkle + scooter away
order.served	Runner carries tray to table, ticket torn off rail	cup on saucer + paper tear
receipt.printed	Printer prints line by line, receipt to spike	thermal print + cut
price.changed	Menu book glows; strike-through and handwritten new price; chip with reason	pen scribble
menu.hidden	"Sold out for now" ribbon	soft stamp
customer.reneged	Patience ring empties, huff cloud, walks out	sigh + door
review.posted	Star floats to HUD (gold, or cracked if low)	sparkle / low thud
equipment.down	Sparks, hazard tape, "out of order" sign	spark zap + low alarm
staff.absent	Empty apron on hook, roster card flips red	phone buzz
po.received	Van outside, boxes slide into fridge	van + box slide
bottleneck.changed	Gauge needle swings to new resource, glow	soft "tick-tick"
investment.delivered	New item drops in with bounce and sparkle	marimba fanfare
kpi.tick (profit milestone)	Odometer rolls, coin pop	ka-ching (pitch ∝ amount)
day.ended	Lights dim, chairs flip onto tables, summary card	music outro

19. Extra ambitious features (our additions)

1. **"Ask Brew" copilot.** An LLM (Claude) with tool access to read-only analytics endpoints answers questions like *"Why is the Rose Latte more expensive right now?"*, *"What should I buy next month?"* and *"Compare yesterday to last Tuesday."* It explains decisions in plain language using the stored `policy_decision.top_factors`. It never takes actions on its own.
2. **Digital-twin calibration from a POS CSV** (§8.8). Any cafe can upload their exports and get a twin of *their* business in minutes.
3. **Live weather integration** via Open-Meteo (free, no key) for the "live" world. Forecasts react to real tomorrow-weather.
4. **Waste-to-donation loop.** Surplus at close goes to a donation crate, ready for a food-rescue partner pickup. Logged as social impact.
5. **Near-expiry "Happy Hour".** RL can trigger automatic markdowns on items close to expiry, turning waste into revenue.
6. **Carbon labels on the menu book** (per-item CO₂e from the recipe BOM), with gentle nudges such as featuring plant-milk options. Measured in the sim as a menu-mix shift.
7. **Staff wellbeing guardrails.** Enforced breaks, max continuous high-load minutes, fair shift rotation and overload alerts, all built into the reward and constraints. A "staff mood" meter appears in the office.
8. **Fair pricing charter.** Hard caps (±10%), minimum time between changes, **no price increases on a "staples" list** (e.g. regular chai, filter coffee), off-peak student discounts, and prices that never change mid-order. Displayed transparently in the menu book.
9. **Replay and time machine.** Scrub any past day, or fork from any moment ("what if the oven hadn't broken at 12:40?").
10. **Customer QR page.** Live menu, live ETA and a "now brewing" board for real customers.
11. **Multi-cafe mode (vision).** Several branches share forecasts and transfer inventory between them, e.g. sending surplus croissants to the busier branch.
12. **Adaptive soundtrack.** Music stems react to kitchen load; an "order up" leitmotif; data sonification of profit.
13. **Accessibility.** Screen-reader announcements of key events (`aria-live`), colour-blind-safe freshness icons (shape + colour), captions for sound cues ("🔔 order up").
14. **Owner mobile digest.** A daily push-style summary card: profit, top bottleneck, recommended action. Rendered in the app (no real push needed).

20. Build plan

20.1 Team split (4 people; adjust to team size)

Role	Owns
P1 Sim & Data	DB schema + Alembic, seed data, simulator engine, personas, kitchen, inventory, delivery, KPIs, Policies A/B
P2 ML & RL	Forecasters, elasticity, CP-SAT/MILP (Policy C), gym env, BC + PPO + adversary, arena eval, bottleneck/advisor
P3 Front end	Store/WS client, rooms (lobby, kitchen, pantry, office, arena), components (ticket, printer, menu book, flap board, fridge), motion
P4 Art, Sound & Story	Food renders, SVG props and paper dolls, sound sourcing + pipeline + AudioDirector mapping, pitch deck, demo script, impact numbers

20.2 Timeline (assuming about 48 hours; scale proportionally)

Hours	Milestone	Exit criteria
0–3	M0 Skeleton	uv project, docker Postgres, Alembic base, FastAPI hello + /ws echo, web/ with tokens and an empty lobby; event schema agreed (§17)
3–12	M1 World runs	Sim engine plus arrivals, personas, choice, kitchen, inventory, delivery; Policy A; one day runs headless in under 1 s; events stream to a bare-bones UI (dots and boxes)
6–16	M1-UI (parallel)	Lobby art in place, ticket rail + printer + menu book components fed by mock events; sound pipeline with the first 20 cues
12–20	M2 Baselines and data	Policies B and C; 120-day synthetic history; LightGBM quantile forecasts + elasticity; arena runner (CLI) with KPIs
16–28	M3 RL	Gym env, BC from C, PPO 1d → 7d curriculum; D-lite beats C on calm scenarios; ONNX export
20–32	M3-UI	Kitchen and pantry rooms, delivery bags, flap board, office (forecast, decisions, bottleneck), arena 2×2
28–36	M4 Chaos + Advisor	Disruptions, adversary, 28-day stage, chaos console; bottleneck + shadow prices + counterfactual advisor; investment shop wiring
36–42	M5 Polish	Transitions, sound mix pass, explanations, impact scoreboard, seeds × 30 eval run, results charts
42–46	M6 Demo lock	Scripted demo world (fixed seed with dramatic moments), offline fallback (pre-recorded event log replay), deck
46–48	Buffer	Rehearse ×3

20.3 De-risking rules

- **Vertical slice first.** Customer → ticket → cook → serve → receipt with ugly visuals before anything pretty.
- **Mock event generator** in the front end so UI work never blocks on the simulator.
- **Replay fallback.** The demo can run from a recorded `events.parquet` → JSON if live compute misbehaves.

- **D-lite before D-full.** Only start D-full once D-lite beats C.
- **Commit trained models.** Policy `.onnx` files (small) are committed, so the demo never depends on training on stage.

21. Risks and mitigations

Risk	Mitigation
RL doesn't beat C in time	BC warm start from C; hybrid D-lite; tune reward; fall back to RL over strategy selection only (still a learned meta-policy)
Simulator too slow	Profile early; <code>__slots__</code> ; vectorised arrivals; cap events; batch WS messages; PyPy/humba only if needed
"Sim-to-real" scepticism	Calibration from POS CSV; ablations; conservative safety shield; human-in-the-loop overrides
Dynamic pricing seen as exploitative	Fair pricing charter, caps, staples list, discounts in off-peak, full transparency on the menu book
UI scope creep	Component checklist with priority tiers (P0 lobby/ticket/printer/menu book/arena; P1 kitchen/pantry; P2 office extras)
Audio licensing	CC0 / Pixabay / Sonnis only; CREDITS.md; no platform jingles or logos
Trademarks (Zomato/Swiggy)	Text labels and generic colours only; no logos; mocked adapters
Live demo crash	Replay mode, fixed seeds, a local-only stack (no internet needed: fonts and libraries vendored)
Overfitting to the adversary	Mixed training (random + adversary + calm); held-out adversary in eval

22. Demo script (5 minutes)

1. **0:00 Hook.** Pink cafe, lo-fi music, the door chimes. *"Every independent cafe runs this hidden optimisation problem every minute of every day. Most solve it with gut feel."*
2. **0:30 The lobby.** The morning rush. Tickets with food renders fly onto the rail, a Zomato order arrives, three iced lattes **batch** with a paperclip snap, the bill printer prints, a rider picks up a "brew" bag. The flap board clatters.
3. **1:15 The menu book.** Flip pages. At the next manager tick, the iced latte goes up ₹10 *"(kitchen 92%, 34°C)"* while a slow-hour waffle discount appears. *"Live, capped, transparent."*
4. **1:45 Kitchen and pantry.** The espresso station is pinned. The fridge opens, a berries lot is about to expire, and the agent triggers a smoothie-bowl happy hour.
5. **2:15 Chaos.** A judge presses **"Barista sick"**. Tickets reshuffle, Swiggy goes to +10 min, pre-prep of cold brew ramps up, paninis get 86'd while the press is short-staffed. A sticky note explains why.
6. **3:00 The Arena.** Same day, four cafes. A's walkouts pile up, B is better, C is good, D wins. Show the profit chart, the 30-seed CI table, and the "where the money came from" waterfall.
7. **3:45 Bottleneck → invest.** *"What's limiting us? Espresso group heads, 7:45–9:30."* Buy a second machine with earned profit. It drops in, the next morning runs, and throughput goes up.
8. **4:20 Impact board.** Waste kg ↓, CO₂e ↓, staff overload minutes ↓, rating ↑, profit ↑. Close with the three impact arguments.

23. Impact: arguments for the judges

Framing: **Brew turns a cafe's hidden operational chaos into decisions that are simultaneously more profitable, less wasteful and kinder to people.** We don't trade one off against the others. The reward function *prices in* waste, staff

overload and customer fairness, so the optimiser improves all three bottom lines together. Every claim below maps to a **metric we measure in the simulator and show live** on the impact scoreboard.

23.1 Economic impact

1. **Profit uplift, proven rather than asserted.** Policy D vs A/B/C on identical days, 30 seeds, paired CIs. This is the core evidence, shown live. Restaurants typically run on thin net margins (commonly cited in the single digits), so a few percentage points of operational improvement can **double** a small cafe's take-home profit.
2. **Smarter capital allocation.** Small owners often invest in the wrong thing, like more tables when the real constraint is the espresso machine. Shadow prices plus counterfactual ROI with confidence intervals mean every rupee goes to the binding constraint. *Avoided bad investments are money saved.*
3. **Channel economics.** Aggregator orders carry high commissions. Brew makes intake decisions on *true* contribution after commission, packaging and the opportunity cost of kitchen time, so a cafe stops losing money on its busiest delivery hours.
4. **Lower cost of goods.** Probabilistic pre-prep and FEFO cut overproduction and expiry write-offs. Near-expiry markdowns convert would-be waste into revenue.
5. **Resilience = less downside.** Adversarial training reduces losses on bad days (staff absence, breakdowns). We report **CVaR** (the average of the worst 10% of days), the metric that decides whether a small business survives a bad month.
6. **Democratising "big-chain" intelligence.** Chains have operations research teams; independents don't. Brew runs on a laptop, needs only POS exports to calibrate, and could be offered as low-cost SaaS. That levels the field for the long tail of independent cafes.
7. **Job-creating growth path.** When the advisor says "*hire a morning barista: +₹X/day, pays back in Y days*", it creates jobs backed by numbers, rather than layoffs driven by panic.

23.2 Environmental impact

1. **Food waste is a climate problem.** The UNEP Food Waste Index (2021) estimated about 931 million tonnes of food wasted globally in 2019, with food service a substantial share, and food loss and waste is commonly estimated at about 8–10% of global greenhouse emissions. Cafes waste through overproduction, expiry and remakes, and **each of these is an explicit variable Brew minimises.**
2. **Mechanisms we measure, each as kg and kg CO₂e per day vs baseline:**
 - **Probabilistic pre-prep:** prep to the right quantile, not "a tray just in case"
 - **FEFO + opened-shelf-life tracking:** use the right lot first
 - **Expiry-aware menu nudges** and happy hours
 - **Fewer remakes** through less chaos and fewer errors under load
 - **Surplus donation** (diverted from bin to people)
3. **Energy.** Batching reduces machine cycles (one panini press cycle for four sandwiches instead of four cycles). Warm-up scheduling avoids idling ovens. The sim tracks kWh from equipment active and idle draw.
4. **Packaging.** Fewer remakes and cancellations mean fewer wasted cups and bags. Packaging is tracked as inventory with its footprint.
5. **Lower-carbon menu nudges.** Per-item CO₂e from the recipe BOM shown on the menu book. The agent can *feature* lower-footprint options (e.g. plant milk) when that is profit-neutral, measured as a menu-mix shift.
6. **Delivery efficiency.** Just-in-time cooking to rider ETA reduces rider idle time at the counter, and batching multiple aggregator orders reduces fragmented handoffs.
7. **Ties to the SDGs.** **SDG 12.3** (halve per-capita food waste at retail and consumer level by 2030), **SDG 13** (climate action).

23.3 Social impact

1. **Staff wellbeing is in the objective, not an afterthought.** Overload minutes (sustained >95% utilisation), enforced breaks and fair shift rotation are built into constraints and reward. Calmer rushes mean less burnout and better retention in an industry known for high turnover. We measure staff overload minutes per day vs baseline.
2. **Fair, transparent pricing.** Unlike opaque surge pricing, Brew's interventions are capped, rate-limited, exclude staples, include off-peak student discounts, and are **displayed live on the menu** with reasons. Customer reference-price fairness is modelled, so exploiting customers is *unprofitable* in our simulator, just as it is in real life.
3. **Better service for everyone, especially those who can't wait.** Persona-aware priorities protect offline customers (commuters, families with kids) whose time pressure is real, without starving delivery customers. Fewer walkouts and fewer cold meals.
4. **Food security.** End-of-day surplus routed to donation (kg donated is tracked). Waste becomes meals.

- 5. **Small-business survival = community survival.** Independent cafes are neighbourhood "third places" that anchor local streets and employ local people. Giving them chain-grade tools helps keep them open.
- 6. **Local suppliers.** The supplier model tracks `is_local` and `distance_km` , and the advisor can show the true cost trade-off of local sourcing.
- 7. **Human-in-the-loop by design.** The owner can override any decision. Every decision is explained, so Brew augments human judgement and doesn't replace it.
- 8. **Accessibility and inclusion** in the product itself: reduced motion, captions for sounds, colour-blind-safe signals. **SDG 8** (decent work), **SDG 2** (zero hunger, via donation).

23.4 Live impact scoreboard (in the Office and the closing slide)

Bottom line	Metric (vs Policy A, same seeds)	Source
Economic	Net profit/day, CVaR ₁₀ %, revenue per labour hour, ROI of recommended investment	<code>daily_kpi</code> , <code>evaluation</code> , <code>counterfactual</code>
Environmental	Waste kg/day, waste ₹, kg CO ₂ e/day, kWh/day, kg donated	<code>stock_movement</code> , <code>daily_kpi</code>
Social	Staff overload minutes, P95 wait (offline), walkouts, avg rating, price changes/day within charter	<code>daily_kpi</code> , <code>review</code> , <code>price_history</code>

All numbers in the pitch come from our own simulation runs and are labelled as simulated. We'll state targets as hypotheses before the run, then show the measured results. That honesty is itself a strong point with judges.

24. Anticipated judge questions

- **"Why RL and not just the solver?"** The solver optimises a known short horizon. RL learns *long-horizon* effects the solver can't see (reputation, loyalty, the value of keeping capacity free for likely dine-in rushes, adapting to disruptions) and learns how to *parametrise* the solver per situation. The ablations show it (D vs C, 28-day vs 1-day training).
- **"Isn't this all synthetic?"** The simulator is a calibratable digital twin: upload a POS export and it fits arrivals, item mix and prep times. The schema serves real cafes unchanged (the `world` abstraction). Live weather can be wired in.
- **"How do you stop the agent doing something crazy?"** A safety shield (action masks plus hard constraints), a fair pricing charter, owner overrides and explanations on every decision.
- **"How does it handle Zomato/Swiggy for real?"** Channel adapters behind a common interface. Today they are simulated; in production they connect via partner integrations or POS middleware.
- **"What's the business model?"** A SaaS subscription per outlet, and/or a share of measured savings. A free tier gives bottleneck analysis from a POS CSV upload.
- **"What's novel?"** The combination: a persona-rich cafe digital twin + probabilistic forecasting + solver + hierarchical RL with adversarial robustness + counterfactual investment advice, all made *legible* through a game-like interface where every decision is seen and heard.

Appendix A: Seed menu (demo cafe "brew")

Category	Items (base ₹)	Station
Coffee	Espresso 140, Cappuccino 190, Latte 210, Rose Latte 260, Iced Latte 230, Cold Brew 220 (prep: concentrate), Filter Coffee 120 (staple)	espresso / cold
Tea and more	Masala Chai 90 (staple), Strawberry Matcha 280, Hot Chocolate 220	bar
Shakes and smoothies	Strawberry Shake 240, Berry Smoothie Bowl 320	blender

Category	Items (base ₹)	Station
Bakery	Croissant 150 (oven, dough prep), Banana Bread 140, Cheesecake 260	oven / display
Savoury	Paneer Tikka Panini 280, Veg Club Sandwich 240, Avocado Toast 320, Fries 160, Pasta Arrabbiata 330	press / fryer / stove
Sweet	Waffle 260, Pancake Stack 280	griddle

Modifiers: milk (dairy / oat +40 / almond +50), extra shot +40, sugar level, ice level, size (R / L +40), add-ons (cheese +30, jalapeños +20), no onion/garlic (Jain), and so on.

Appendix B: Example scenario config

```
key: rainy_delivery_surge
weather_profile: {temp_c: [24, 27], rain_mm_per_h: {12-16: 6, 19-22: 8}}
calendar_events: [{kind: cricket_match, start: "19:30", end: "23:00", delivery_mult: 1.6}]
persona_mix_override: {delivery_home: 1.5, leisurely: 0.6}
disruption_schedule:
  - {kind: rider_shortage, start: "20:00", end: "21:30", eta_mult: 1.8}
  - {kind: staff_late, target: {role: barista}, start: "07:00", delay_min: 45}
```

Appendix C: Policy interface

```
class Policy(Protocol):
    code: str # "A".."E"
    def reset(self, world: WorldView, seed: int) -> None: ...
    def act(self, obs: Observation, point: DecisionPoint) -> Action: ...
    def explain(self, decision_id: UUID) -> Explanation: ...
```

Appendix D: Glossary

- **FEFO:** first-expiry, first-out.
- **86:** remove an item from the menu temporarily.
- **Newsvendor:** single-period stocking under uncertain demand.
- **CVaR:** expected value of the worst α % of outcomes.
- **RARL:** robust adversarial reinforcement learning.
- **Shadow price:** marginal value of one more unit of a constrained resource.
- **CRN:** common random numbers, the same randomness across compared policies.
- **JIT:** just-in-time.
- **BOM:** bill of materials (recipe).